Assignment 1 - RoboWorks

Overview

This assignment is due at 11:55 pm on Thursday the 2nd of April 2026 as detailed in Moodle. Submission of this assignment will be via this activity on Ed.

This assignment is worth 25 marks and is worth 25% of your grade. This assignment is an individual assessment and is not to be completed with the assistance of anyone not on the FIT9136 S1 2026 teaching team.

This assignment covers content from weeks 1-4, inclusive.

Before starting this assignment you must review the slide on Academic Integrity.

Your assignment must be written in the ED platform using only the workspaces provided for the assignment. Code is not to be copied and pasted from sources that are not the given ED workspace.

At any point, you may seek assistance from your TAs in your applied class or via a private post on the forum. Any questions of a general nature may be asked using a public post for the benefit of other students.

Academic Integrity

Plagiarism and collusion are viewed as serious offences. All submitted code will be subjected to a similarity checker, and any submissions determined to be similar to a submission from a current or past student will be investigated further. The outcome of the decision pertaining to plagiarism and/or collusion will be determined by the faculty administration. To ensure compliance with this requirement, be sure to do all your coding in the Ed workspace environment and do not copy and paste any code into the workspace environment.

In cases where plagiarism or collusion has been confirmed, students have been severely penalised, ranging from losing all marks for an assignment, to facing disciplinary action at the Faculty level. Monash has several policies in relation to these offences and it is your responsibility to acquaint yourself with these.

Assessment and Academic Integrity Policies and Procedures (https://www.monash.edu/students/study-success/academic-integrity)

In this assessment, you must not use generative artificial intelligence (GenAI) to generate any materials or content in relation to the assessment task. It is prohibited to use GenAI for reasoning or to gain insight into a task, including for the purpose of translation. You should instead be using a dedicated translation service that will not attempt to interpret the question for you. We consider genAI to be, but not limited to, ChatGPT, Claude, Gemini, DeepSeek and GitHub Copilot.

Generative artificial intelligence also takes the form of code completion systems that are built into some editors. It is up to you to make sure that you are not using any generative artificial intelligence.

You are responsible for all code submitted as a part of your assignment.

Code Quality

Some items we consider when marking code quality:

- Using meaningful variable names

```
def calculate_total(a, b, c):
    x = a + b
    y = x * c
    return y

def calculate_total(price, tax, quantity):
    subtotal = price + tax
    total_cost = subtotal * quantity
    return total_cost
```

Consider the following:

- Clear and meaningful loop and conditional statements
- Meaningful comments (From task 3 onwards)
    - Comments should be used to explain the purpose of sections of code **not** to restate what the code in English is.

Good comments:

```
def calculate_discount(price, discount_rate):
    # Ensure discount_rate is treated as a percentage (0–100)
    if discount_rate > 1:
        discount_rate = discount_rate / 100

    # Apply discount
    return price - (price * discount_rate)
```

Over commenting (bad):

```
def calculate_discount(price, discount_rate):
    # Take price
    price = price

    # Check if discount_rate is greater than 1
    if discount_rate > 1:  # if the discount rate is more than 1
        # Divide discount_rate by 100
        discount_rate = discount_rate / 100

    # Multiply price by discount_rate
    discount = price * discount_rate

    # Subtract discount from price
```

```
    return price - discount
```

- Maintaining consistent indentation and spacing
- Avoiding unnecessary complexity
    - Complexity in this unit simply refers to is the code straightforward to follow without complications. Some examples of overly complex code would be portions of the code having no real effect.
    - There is another definition of complexity this is not taught or assessed in this unit, you are not assessed on the speed / efficiency of your code so long as it runs without error within a reasonable time period (if it takes too long to run the auto marker will give you an error).

The following is a an over exaggeration but in general we do have the ability to make some deductions should there be room for improvement but given it's the first assignment of an entry level programming unit we are understanding.

```
def greet_user():
    # Ask for name with redundant type conversions
    raw_input_value = input("What is your name? ")
    name_as_string = str(raw_input_value)   # redundant, already str

    # Extra step: copy into another variable
    confirmed_name = "{}".format(name_as_string)  # unnecessary .format

    # Create unused variable
    unused_variable = confirmed_name.upper()

    # Pointless boolean check
    is_valid = True if len(confirmed_name) > 0 else False

    if is_valid:
        # Overly verbose string concatenation
        greeting = "Hello, " + confirmed_name + "!"
        return greeting
    else:
        return "You didn't enter a name."

# Reasonable implementation
def greet_user():
    name = input("What is your name? ")
    if name:
        return f"Hello, {name}!"
    else:
        return "You didn't enter a name."
```

- Structuring your code logically and concisely
- Avoiding magic strings / magic numbers:
    - These are values that have no explanation or context.
- Docstrings should be provided for all functions, the preferred format is:
    - Function description

- Function parameters with types
- Function returns with types

An example docstring is provided below:

```
def function_with_pep484_type_annotations(param1: int, param2: str) -> bool:
    """Example function with PEP 484 type annotations.

    Args:
        param1: A short description of the first parameter.
        param2: A short description of the second parameter.

    Returns:
        The return value. True for success, False otherwise.

    """
    pass
```

How will my work be assessed?

In general, your work will be assessed in two ways.

1. **Functionality:** The functionality of your code will be assessed using a series of test cases. These test cases will not be available to you before submission. A series of public test cases will be given to allow you to work with the input and output as specified in the assignment.
2. **Code quality:** Code quality refers to the the design of your code. This includes, but is not limited to variable names, comments, using "snake" case, human readability, and other categories. For more information on commenting see this post (Not all of these examples will be applicable to you at this time, but you should understand each of the examples by the end of the semester).

Functionality (Automated testing)

We will provide you with a series of public test suites that will be useful for understanding the expected input and output. We are not trying to trick you. Private tests are designed to ensure that your solution has the required functionality and ensures that you have engaged critically with the work.

Testing well does not guarantee that your assignment will receive full marks for functionality. For example, if your code returns the correct result while ignoring details that are required by the specification, then you will not be eligible for the marks that are associated with the test cases.

Code quality

Code quality will be marked based on the readability and clarity of your coding solutions. This has nothing to do with functionality. A code base with full functionality can earn no marks for code quality, if the code is poorly written.

Documentation

Any functions that you have created must have documentation. This is regardless of whether we require the function or you decide to create a new function.

Inline comments and Variable naming

Unlike the documentation, you are expected to use in-line comments throughout your program to make sure it is clear to the reader. The amount of comments necessary depends on how clear the code is by itself. Therefore, a good habit is to write clear code, so that the amount of in-line comments required is reduced. This will also reduce the number of bugs created.

Comment your code as you write it rather than after reaching a final solution, as it is a lot easier to

remember why certain decisions were made and the thought process behind them when the code is fresh in your mind.

Ensure your variable names are meaningful and concise so that your code is understandable at first read. When a reader is going through your program, it should read like a self-explanatory story, using in-line comments to explain the logic behind blocks of code and any additional information that is relevant to the functionality of the code.

Submission Requirements

- Your final submission will be the Python files from every task on the Ed lesson. Do **NOT** change the name of these files. Code written in other files will not be eligible to receive marks.
- You are not allowed to write any `import` statements as part of this assignment.

Once you have completed all tasks of the assignment to your satisfaction, clicking the blue `Submit` button in the top right corner of the Ed lesson will officially perform your final submission for the assignment. You are only allowed to make **a single submission** of the assignment so please *double check* that you have **completed all tasks** before submitting!

To test your submission you can use the "Test" button at the bottom left hand side of the screen. This button can be used to test your submission as many times as you would like, there is no limit. However, as mentioned above, you may only submit once.

RoboWorks - Programming the Robot Factory

You are a junior developer at **RoboWorks**, a robot construction company. Across three tasks, you'll build from simple calculations to a modular mini-API for assembling robots. Each task includes **unassessed subtasks** that build skills; **only the final task of each section is assessed**.

Task Weighting:

- Task 1 — Parts & Pricing Basics: **25%** (assessed: final section only)
- Task 2 — Assembly Line Simulator: **35%** (assessed: final section only)
- Task 3 — Modular Robot Builder API: **40%** (assessed: final section only)

Note on subtasks: You can complete the **subtasks** in any order, or not at all, to build up to your your solution. They are **not individually assessed**. **Only the final task for each section is graded**. However, they do reflect your submission and as such are subject to the same academic scrutiny. Under no circumstances should you share code relating to any task or subtask in this assignment with another student or anyone outside of the teaching team. Likewise, you should not use any code that has been shared with you or is the product of a generative AI.

Task 1 - Parts and Pricing

Your new boss is tired of his sales associates under quoting the price of robot construction. Their first task is to create a console program that asks the user for quantities of known robot parts, computes a subtotal, applies a discount rule, and prints a clear, aligned quote summary.

Subtask 1.1 The price of motors (Not assessed)

Your boss, in his wisdom, has tasked you with writing a small script that will allow you to compute the price of an order of motors. The script should prompt the user for the number of motors required. The script should then output the line total for the cost of the motors, rounded to two decimal places. For the moment, there is no need to worry about if the quantity being purchased is an positive integer.

Examples:

```
Welcome to the motor line item test script.

Motor price: $52.99

How many motors would you like to buy: 5
Total price: $264.95
```

```
Welcome to the motor line item test script.

Motor price: $52.99

How many motors would you like to buy: 2.5
Total price: $132.48
```

```
Welcome to the motor line item test script.

Motor price: $52.99

How many motors would you like to buy: -1
Total price: $-52.99
```

All variables should be stored in accordance with good programming conventions and strings should be printed using variables rather than hard-coded values.

For this subtask, you may assume that all input is sensible. No validation is required.

Subtask 1.2 Looking for inventory (Not assessed)

To extend your skills, you boss has decided that you should consider integrating the company inventory. You boss wants to you write a small script that represents the following table as a pair of aligned list, one for product name and one for price. The program should display each product, along with the matching price, by iterating over the lists at the same time.

```
-----------------------
Product Name | Price
-----------------------
motor        |  $49.99
sensor       |  $15.75
frame        | $120.00
cpu          |  $85.50
```

Do not overthink this exercise, it is a simple exercise designed to help you understand the required lists.

No need to do alignments (i.e., formatting with spaces) in your outputs.

Subtask 1.3 Bargains galore (Not assessed)

Your boss wants you to undertake one last exploration... A conditional discount calculator. You are to write a simple program that prompts the user for the end sales price and applies a conditional discount. The discounts are as follows: 15% for orders over $1500.00, 10% for orders over $1000.00, and 5% for orders over $300.00.

All discounted amounts should be displayed with two decimal places and a dollar sign.

Examples:

```
Welcome to the discount calculator.

Enter sales amount(dollars): 10.00
Discounted price: $10.00
```

```
Welcome to the discount calculator.

Enter sales amount(dollars): 300
Discounted price: $300.00
```

```
Welcome to the discount calculator.

Enter sales amount(dollars): 301.00
Discounted price: $285.95
```

```
Welcome to the discount calculator.

Enter sales amount(dollars): 2000
Discounted price: $1700.00
```

```
Welcome to the discount calculator.

Enter sales amount(dollars): 1024.00
Discounted price: $921.60
```

For this subtask, you may assume that all input is sensible. No validation is required.

Task 1 - Quote Calculator (Assessed)

Your boss now wants you to take what you have learned a create a functional quote calculator for the simple electronics division. Your boss would like you to formalise what you did in subtask 1.2 by creating two aligned lists `product_name`, containing a list of product names, and `product_price`, containing a list of product prices. You should populate this list using the table below.

```
----------------------
Product Name | Price
----------------------
motor        |  $49.99
sensor       |  $15.75
frame        | $120.00
cpu          |  $85.50
```

You must conform to the variable naming given in the task. In testing, we will manipulate these lists. If they are not named correctly, then you will fail the tests and lose the marks.

Your program will then prompt the user for a desired quantity of each part from the list `product_name`. Using this information you will compute the line total for each item, the subtotal for all items, the discount (in dollars, even if $0.00), and the total price.

Examples:

```
Welcome to the RoboWorks Quote Calculator.

For each product below, please specify your required quantity.

motor: 1
sensor: 2
frame: 3
cpu: 4

Please see your quote below.

motor: 1 x $49.99 = $49.99
sensor: 2 x $15.75 = $31.50
frame: 3 x $120.00 = $360.00
cpu: 4 x $85.50 = $342.00

Subtotal: $783.49
Discount: $39.17
Total: $744.32
```

```
Welcome to the RoboWorks Quote Calculator.

For each product below, please specify your required quantity.
```

```
motor: 2
sensor: 1
frame: 0
cpu: 0

Please see your quote below.

motor: 2 x $49.99 = $99.98
sensor: 1 x $15.75 = $15.75
frame: 0 x $120.00 = $0.00
cpu: 0 x $85.50 = $0.00

Subtotal: $115.73
Discount: $0.00
Total: $115.73
```

```
Welcome to the RoboWorks Quote Calculator.

For each product below, please specify your required quantity.

motor: 5
sensor: 6
frame: 15
cpu: 8

Please see your quote below.

motor: 5 x $49.99 = $249.95
sensor: 6 x $15.75 = $94.50
frame: 15 x $120.00 = $1800.00
cpu: 8 x $85.50 = $684.00

Subtotal: $2828.45
Discount: $424.27
Total: $2404.18
```

Task 2 - Assembly Line Simulator

After your success building the quote calculator, your boss decides to re-tasked you with working on the new robot assembly line program. This program will be central to managing the assembly line and it looks like you are the best person for the job.

Subtask 2.1 Inventory printout (Not assessed)

You decide to take the lead and want to write your first test program. You hope that this first program will help you develop your inventory structure. To test this you will need to create three aligned lists: `part_name`, `part_stock`, and `part_cost`. The function of each list is described below:

- `part_name`: This list contains the part names, stored as strings.
- `part_stock`: This list contains the number of this specific part that is currently in stock.
- `part_cost`: This list contains the unit cost to the company of each part.

Remember to follow the naming convention for the testing to work correctly.

The following table gives the required values:

```
| Part Name   |   Part Stock | Part Cost   |
|-------------|--------------|-------------|
| motor       |            5 | $30.50      |
| sensor      |            7 | $10.20      |
| frame       |           10 | $60.00      |
| cpu         |            2 | $45.95      |
```

The program should display each product, along with the total dollar value of the items in inventory, by iterating over the lists at the same time.

Example on the data given above:

```
motor: $152.50
sensor: $71.40
frame: $600.00
cpu: $91.90
```

Do not overthink this exercise, it is a simple exercise designed to help you understand the required lists.

Subtask 2.2 Model definition and feasibility checks (Not assessed)

Your boss wants you to start thinking about the construction of the `R2` model robot. The `R2` requires 3 motors, 2 sensors, 3 frames and 1 cpu. You decide to define a model, such as the `R2`, as an aligned list of parts called `model`. Write a program that uses the lists defined in task 2.1 and a definition of a model to decide if the model can be reasonably constructed, given the parts on hand. This program should not modify any of the lists.

Have the program print `True` if the model can be constructed from the parts on hand and `False` otherwise. The program should have no other output.

For example, given the table in task 2.1, the model `R2` can be constructed.

When given the following table, the `R2` cannot be constructed.

```
| Part Name   |   Part Stock | Part Cost   |
|-------------|--------------|-------------|
| motor       |            5 | $30.50      |
| sensor      |            7 | $10.20      |
| frame       |           10 | $60.00      |
| cpu         |            0 | $45.95      |
```

This is because the company does not have enough cpu parts in stock.

Remember to follow the naming convention for the testing to work correctly.

Subtask 2.3 One-unit build (Not assessed)

Given the model defined in task 2.2 and the table of goods as defined in table 2.1. Check if the construction of `R2` is feasible. If it is, then decrement the required stock from the part_stock list and return the cost of constructing the `R2` model, in parts alone. If the model cannot be created, then report it.

We can decrement a list the same way as we decrement an integer variable, we just need to index the correct position in the list first.

Examples:

```
The model can be constructed and will cost $337.85
```

```
The model cannot be constructed.
```

Remember to follow the naming convention for the testing to work correctly.

Task 2 - Order Queue Processor (Assessed)

Your boss looks over the programs you have written and decides that you are ready to write the Order Queue Processor.

As part of this program, you are given an updated stock table.

```
| Part Name   |   Part Stock | Part Cost   |
|-------------|--------------|-------------|
| servo       |           42 | $38.79      |
| lidar       |           28 | $245.30     |
| motor       |           63 | $52.99      |
| sensor      |           29 | $21.45      |
| gyroscope   |           54 | $132.88     |
| gearbox     |           51 | $310.60     |
| regulator   |           95 | $27.14      |
| controller  |           77 | $89.53      |
```

This stock table should be stored in aligned lists that represent the current stock, just as in task 2.1-3. As well as these variables, you should also represent the following table in a variable called `models`.

```
| Part       |   R1 |   R2 |   R3 |   R4 |
|------------|------|------|------|------|
| servo      |    4 |    7 |    3 |    1 |
| lidar      |    6 |    0 |    6 |    6 |
| motor      |    5 |    4 |    2 |    3 |
| sensor     |    4 |    4 |    7 |    6 |
| gyroscope  |    0 |    0 |    7 |    4 |
| gearbox    |    2 |    6 |    4 |    5 |
| regulator  |    2 |    5 |    4 |    2 |
| controller |    7 |    5 |    4 |    7 |
```

The variable `models` should contain a dictionary where the key of the dictionary is the name of the model and the value is the list from subtask 2.2-3, that is aligned with the stock lists, and contains the number of each part required to construct the model.

You should also have a variable called `queue` which contains the order queue. The order queue is a list of tuples of size two where the first element is the name of the model to be built, and the second element is the number of that given model that are required. A sample order queue is specified below.

```
("R4",2)
("R1",2)
("R3",1)
("R2",4)
("R1",1)
("R4",2)
```

```
("R2",3)
```

The program then processes the orders, one model at a time. Before building each robot, you must check the feasibility across all parts. If the robot can be feasibly constructed, given the inventory, then the stock is decremented and the build report (which tracks how many robots were built and the expected revenue) is updated. If the robot cannot be constructed, then the robot is put on backorder.

The program then prints a report containing the number of built robots, per model, and the total revenue. The report should also contain the number of each model on backorder and the remaining inventory.

Examples:

```
Constructed units
R1: 2
R2: 2
R3: 0
R4: 2

Total cost: $21719.82

Backorder
R1: 1
R2: 5
R3: 1
R4: 2

Inventory
servo: 18
lidar: 4
motor: 39
sensor: 1
gyroscope: 46
gearbox: 25
regulator: 77
controller: 39
```

Remember, the variable names of the input data must be consistent. Otherwise the tests will fail and the assignment will lose marks.

Task 3 - Modular Robot Builder API

Your boss is extremely impressed with the work you have done and would like to make the systems available to the company as a whole. So do this he requires 2 things:

1. An API (Application programming interface),
2. and a CLI (Command line interface)

An API is a way for other developers to interact with you programs. It is a series of functions and modules that can be used by programmers outside your immediate team. This involves organising and changing your code to make it modular and reusable.

A CLI is a command line interface. This is a way for users in interact with your program through the use of the command line. For example, someone doing a quick quote might want access to the quoting functionality. Subtasks 3.1-2 will help you understand how to build both of these interfaces.

Subtask 3.1 Dictionary model setup (Not assessed)

It becomes clear that your previous setup will not be particularly useful for designing your modular API. To fix this, you decide to look into using dictionaries to deal with relational data. This means that you will no longer have to deal with the issues posed by aligned lists.

You must create three new variables:

- `PRICE_CATALOG`: The variable `PRICE_CATALOG` is a constant that contains a dictionary that maps the part name to the cost of the part.
- `MODELS`: The variable `MODELS` is a constant that contains a dictionary that maps the model name to a dictionary that maps the part codes (name) to required number of that part.
- `inventory`: The variable `inventory` contains a dictionary that maps the part code (name) to the current number of that part that is in stock.

To test this new setup, you should use the data from task 2 and calculate the cost of each model in the models dictionary.

Example:

```
R1: $3279.90
R2: $3016.24
R3: $4483.54
R4: $4563.77
```

Subtask 3.2 Core functionality (Not assessed)

In order to understand how to build an API, you decide that you should start to use functions to model the core functionality. You decide to create four core functions:

In the following functions, the variables `models`, `catalog`, and `inventory` are the local variables for accessing the variables as defined in task 3.1.

1. `get_model_cost(model, catalog, models)`: This function should return the cost required to construct the `model` given the `catalog` and `models` dictionaries. These variables should be local to the scope of the function during the function call, not global to the program.
2. `can_build_one(model, inventory, models)`: This function should return `True` if the model can be build, given the available inventory. It should return `False` otherwise.
3. `build_one(model, inventory, catalog, models)`: This function should decrement the inventory and return the cost of the model. If the model cannot be built then the inventory should not be modified and a cost of `$0.00` should be returned.
4. `apply_discount(total)`: This function should return the new total after applying the discount as defined in task 1.

To complete these functions appropriately your should write docstrings and type annotations for the functions. The type annotations should include return types.

Your function signatures need to match the given function signatures. If they do not, then the tests will fail.

Task 3 - Mini-API with CLI (Assessed)

Now that you have completed the warm-up, you decide that it is time to put all your hard work together. To do this you need only extend the functionality already described in tasks 3.1-2. You are told to use the current data as specified for by your boss in task 1.

The API

You decide on the following API:

1. `get_model_cost(model, catalog, models)`: This function should return the cost required to construct the `model` given the `catalog` and `models` dictionaries. These variables should be local to the scope of the function during the function call, not global to the program.
2. `can_build_one(model, inventory, models)`: This function should return `True` if the model can be build, given the available inventory. It should return `False` otherwise.
3. `build_one(model, inventory, catalog, models)`: This function should decrement the inventory and return the cost of the model. If the model cannot be built then the inventory should not be modified and a cost of `$0.00` should be returned.
4. `process_order(model, count, inventory, catalog, models)`: This function should attempt to fill an order and in doing so return a tuple of size 3 containing the number of constructed models, the number of models to be placed on backorder, and the cost of the constructed models.
5. `apply_discount(total)`: This function should return the new total after applying the discount as defined in task 2.

CLI Menu

You decide to build a thin CLI wrapper. This should allow users to interact with the system and act on the data that was specified. Your program should display the following menu:

```
1) Show models and costs
2) Attempt order
3) Show inventory
0) Exit
```

The menu should ask the user to input an integer. That integer is used to carry out the task. In the case of option (2) the user should be prompted for more information, first the model as a string and then the quantity as an integer.

All integers should have their bounds checked. You do not need to perform type validation. If we tell you that the input has a specific type, you can assume that to be the case.

Example:

```
1) Show models and costs
2) Attempt order
3) Show inventory
0) Exit

Please enter an integer between (0-3): 1
R1: $3279.90
R2: $3016.24
R3: $4483.54
R4: $4563.77

1) Show models and costs
2) Attempt order
3) Show inventory
0) Exit

Please enter an integer between (0-3): 2
Please enter a model number: R2
Please enter the number of R2 units you would like: 7

Attempting to order models...

R2 order details.
Units built: 6
Units on backorder: 1

Subtotal: $18097.44
Discount (dollars): $2714.62
Total: $15382.82

1) Show models and costs
2) Attempt order
3) Show inventory
0) Exit

Please enter an integer between (0-3): 3
Current inventory:

servo: 0
lidar: 28
motor: 39
sensor: 5
gyroscope: 54
gearbox: 15
regulator: 65
controller: 47

1) Show models and costs
2) Attempt order
3) Show inventory
0) Exit
```

```
Please enter an integer between (0-3): 0
```

The code should demonstrate good separation of responsibility. This means that the API functions should not interact with the console, so no input or print statements. You are welcome to use as many functions for the CLI as you would like, but these functions must use the API wherever possible.

Marking Criteria

Each task will be marked against the following criteria.

Change log

This is your change log. Hopefully this is small.

1. Error related to gearbox count in task 2. (Fixed 11 Mar 5:25 pm)
2. Error related to queue in task 2. (Fixed 11 Mar 5:26 pm)
3. Error related to inclusion of catalog in the function `build_one` in task 3.2. (Fixed 11 Mar 5:33 pm)
4. Missing a blank line after operation 2's outputs (i.e., between `Total: $15382.821` and `Show models and costs`) (Fixed 13 Mar 6:07 pm)
5. Missing clarifications on the format of outputs for Task 1.2 (Fixed 13 Mar 6:11 pm, see the blue info field)
6. Task 1 missing decimal precision in the example output. Specifically in the in the cost of the `frame` and `cpu`. (Fixed 16 Mar 11:55 pm)
7. Task 2 wrong wording. In the printed outputs, `Total revenue` should be `Total cost` (Fixed 17 Mar 10.16am)
8. Task 3 has been updated with bug fixes and more user-friendly messages. (This applies after 2:58 pm, 17th March)
9. Subtask 3.2 and Task 3 reference task 2 with respect to the discount calculation. This has been corrected to task 1. (Fixed 19 Mar 09:58 am)
10. Added clarification to the task 3.1 dictionary setup part name/code. (Added 20 Mar 11:32 am)
11. Public Test Case 3.4: you may fail in this case due to the use of the main guard (i.e., __name__ == "__main__"). Now, test cases should be passed no matter whether w/ or w/o main guard is used or not. (This applies after 21 Mar 02:35 pm)